CSC 2105::Data Structure

Stack Application

Fahad Ahmed

Assistant Professor, Department of Computer Science

Faculty of Science & Information Technology

American International University-Bangladesh (AIUB)

mashiour@aiub.edu

APPLICATIONS OF STACK

- Syntax parsing, Expression evaluation and Expression conversion.
- Banking Transaction View
  - You view the last transaction first.
- Backtracking and implementation of recursive function, calling function.
- Towers of Hanoi
- Inventory Systems like Issuing students the Multi-meters… you will be issued the most recently returned item likely

ALGEBRAIC EXPRESSION

- An algebraic expression is a legal combination of operands and the operators.
- Operand is the quantity (unit of data) on which a mathematical operation is performed.
- Operand may be a variable like `x,y,z` or a constant like `5,4,0,9,1` etc.
- Operator is a symbol which signifies a mathematical or logical operation between the operands. Example of familiar operators include `+,-,*,/,^,%`
- Considering these definitions of operands and operators now we can write an example of expression as: **`x + y * z`**

INFIX, POSTFIX AND PREFIX EXPRESSIONS

- **INFIX**: The expressions in which operands surround the operator, i.e. operator is in between the operands. e.g. `x+y, 6*3` etc. The infix notation is the general way we write an expression.
- **POSTFIX**: Also Known as Reverse Polish Notation (RPN). The operator comes after the operands, i.e. operator comes post of the operands, so the name postfix. e.g. `xy+, xyz+*` etc.
- **PREFIX**: Also Known as Polish notation. The operator comes before the operands, i.e. operator comes pre of the operands, so the name prefix. e.g. `+xy, *+xyz` etc.

INFIX

- To our surprise INFIX notations are not as simple as they seem specially while evaluating them. To evaluate an infix expression we need to consider Operators' Precedence and Associative property
- For example expression `3+5*4` evaluate to

  `32 = (3+5)*4` or

  `23 = 3+(5*4)`

- Operator precedence and associativity governs the evaluation order of an expression.
  - An operator with higher precedence is applied before an operator with lower precedence.
  - Same precedence order operator is evaluated according to their associativity order.

Operator Precedence and Associativity

Precedence	Operator	Associativity
1	::	Left-to-right
2	() [] . -> ++ --	Left-to-right
3	++ -- ~ ! sizeof new delete	Right-to-left
	* &	
	+ -	
4	(type)	Right-to-left
5	.* ->*	Left-to-right
6	* / %	Left-to-right
7	+ -	Left-to-right
8	<< >>	Left-to-right

Precedence	Operator	Associativity
9	< > <= >=	Left-to-right
10	== !=	Left-to-right
11	&	Left-to-right
12	^	Left-to-right
13	|	Left-to-right
14	&&	Left-to-right
15	||	Left-to-right
16	?:	Right-to-left
17	= *= /= %= += -= >>= <<= &= ^= |=	Right-to-left
18	,	Left-to-right

Infix Expression Is Hard To Parse

- Need operator priorities, tie breaker, and delimiters.
- This makes the evaluation of expression more difficult than is necessary for the processor.
- Both prefix and postfix notations have an advantage over infix that while evaluating an expression in prefix or postfix form we need not consider the Precedence and Associative property.
- The expression is scanned from user in infix form; it is converted into prefix or postfix form and then evaluated without considering the parenthesis and priority of the operators.
- So, it is easier (complexity wise) for the processor to evaluate expressions that are in these forms.

EXAMPLES OF INFIX TO PREFIX AND POSTFIX

Infix	PostFix	Prefix
A+B	AB+	+AB
(A+B) * (C + D)	AB+CD+*	*+AB+CD
A-B/(C*D^E)	ABCDE^*/-	-A/B*C^DE

A - B / (C * D ^ E)

A - B / (C * F)

A - B / G

A - H

I

ABCDE^ * / -

ABCF * / -

ABG / -

AH -

I

D^E

C * F

B / G

A - H

- A / B * C ^ DE

- A / B * C F

- A / B G

- AH

I

D^E

C * F

B / G

A - H

Converting Infix to Postfix

Infix Expression: 2*6/(4-1)+5*3

```
Add ')' to the end of Infix;     Push( '(' );
do{
  OP = next symbol from left of Infix;
  if OP is OPERAND then EnQueue( OP );
  else if OP is OPERATOR then{
    if OP = '(' then Push( OP );
    else if OP = ')'  then{
        Pop( StackOperator );
        while StackOperator != '(' do{
        Enqueue( Stackoperator );
        Pop( StackOperator );
        }`
    }else{
    while Precedence( OP ) <= Precedence( TopElement( ) ) do{
        Pop( StackOperator );
        Enqueue( StackOperator );
    }
    Push( OP );
    }
}while !IsEmpty( );
```

Infix	2	*	6	/	(	4	-	1	)	+	5	*	3	)

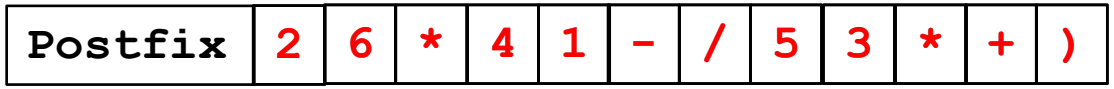
Postfix	2	6	*	4	1	-	/	5	3	*	+

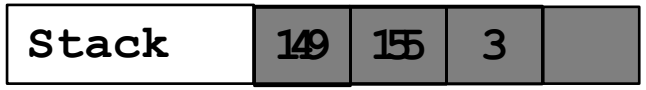
Stack					

End of Expression

- ≥ StackTop(( )

Evaluating Postfix Expression










```
Postfix Expression: 26*41-/53*+
EnQueue( ')' );
while ( FrontElement( ) != ')' ) do{
     DeQueue( OP );
     if OP is OPERAND then Push( OP );
     else if OP is OPERATOR then{
          Pop( OperandRight );
          Pop( OperandLeft );
          Push( Evaluate( OperandLeft, OP, OperandRight ) );
     }
}
Pop( Result );
cout << Result;
}
```